P vs NP Grassroots:
Ground-Up Background Theory for a Structured Switching Route

May 21, 2026

Abstract

This note records the *grassroots* side of the current Lean work around a proposed route to $P \neq NP$. It is intentionally not the blocker note. The companion crux-first analysis lives separately in `pnp_crux.tex`.

The goal here is narrower and more constructive: build reusable background theory from the ground up, using mathlib and existing Mettapedia infrastructure wherever possible, so that if the high-level proof strategy can be repaired, it lands in a clean machine-checked framework rather than in ad hoc prose.

The current grassroots state now includes explicit encoded hypothesis classes, finite empirical-risk minimization, finite deceptive-sample bounds, threshold recovery lemmas, a rational uniform-rate layer, and explicit uniform success bounds for exact ERM recovery under finite sampling. The success layer now also exposes non-deceptive sample-rate lower bounds directly, so downstream arguments can use either the realized-target recovery theorem or the target-independent complement of the deceptive region. It now also includes the first weighted finite-sampling steps beyond uniform draws: an exact one-predictor agreement law and a family-level weighted union bound over finite encoded classes, together with weighted exact-recovery lower bounds for ERM. It also now packages the exact *same-route interface*: a repair that still intends to use switching plus ERM must produce a bit-encoded family over an available input surface and therefore inherits the current finite weighted recovery theorem. It also now isolates the exact post-switch visible surface and proves a clean finite-summary kernel: any switched family that truly factors through a finite summary surface automatically inherits an explicit truth-table bit budget equal to the cardinality of that summary type. It now also ties the four current exact-surface affine candidate classes directly to the weighted recovery backbone: once the target lies in one of those classes, the finite-sampling lower bound is available immediately with the matching explicit bit budget. It now also includes a probability-free finite-count conditional-independence API: exact independence, approximate tolerance, output-fiber certificates, and finite-output max defects are all connected by small reusable Lean theorems. It now also turns the balanced-fiber fork into grassroots source-erasure infrastructure: balanced finite-coin output fibers are exactly neutrality for all decidable output predicates, and such erasure forces every Boolean output decoder to have exactly half aggregate accuracy. This is now connected back to the finite-count independence API: on the sample space Bool $\times$ Coin, predicate neutrality is equivalent to exact finite-count independence from the source bit, and balanced fibers are equivalent to zero output defect. The quantitative form is also explicit: each output fiber's symmetric defect is exactly |Coin| times its true/false coin-fiber imbalance, so an approximate-independence tolerance is precisely a uniform bound on those scaled imbalances. It now also isolates a sharper shared-basis regime: if one fixed affine feature extractor is reused across the switched family and only the downstream combiner varies, then the code budget collapses to the combiner alone. The Meredith-side limit is now considerably sharper. The currently formalized weakness bridge exports only global evidence scalars, so any predictor factoring only through those summaries is constant on the whole state space. The current exact HML layer is infinitary by default: if the certified minimal-context

type is infinite, then even the depth-1 fragment is infinite, and the concrete rho instance already realizes that case. And one layer below exact HML, the concrete rho present-moment surface already realizes arbitrary finite membership patterns on a natural probe slice: a single-output probe state against a flat bag of input guards records exactly which channels are present in the environment. On the semantic side, it now includes a depth-bounded GSLT/HML observational kernel, a bridge from bisimulation quotients to quantale weakness, and a first concrete minimal-context instance for the rho calculus. This isolates the exact positive shape that a successful switching theorem would need: not mere locality, but locality together with a concrete compression theorem, a clean finite learning interface, and an explicit observational/evidence semantics.

Contents

1 Scope

The October 2025 manuscript *"P != NP: A Non-Relativizing Proof via Quantale Weakness and Geometric Complexity"* proposes a route from a structured Unique-SAT ensemble to a distributional lower bound, and then to a contradiction with the upper bound that would follow from $P = NP$.

This document does *not* ask where that route fails. That is the job of the companion crux note. Instead it asks a different question:

> What background theory would we want in Lean even if the final proof still needs repair?

That question leads to a program with three design constraints:

1. Prefer general infrastructure over one-off rebuttal lemmas.
2. Reuse mathlib computability and existing Mettapedia semantics rather than inventing parallel foundations.
3. State optimistic bridge theorems in a way that makes missing hypotheses explicit, so later fixes can slot into them cleanly.

2 Existing Lean Base

Several ingredients already exist in the local formalization and are useful for a ground-up PNP effort.

Module	Grassroots role
Mathlib.Computability.Partrec	partial recursive computa
KolmogorovComplexity/Basic.lean	universal algorithm and p
Hyperseed/Ultrainfinitism.lean	observer-relative available
Hyperseed/Basic.lean	closure/cascade interfaces
GSLT/Logic/LogicalMetric.lean	finite-depth observational
GSLT/Meredith/WeaknessBridge.lean	operational distinguishab
GSLT/Meredith/RhoMinimalContext.lean	first concrete minimal-co
PNP/HypothesisClass.lean	encoded classifier families
PNP/FiniteERM.lean	empirical-risk minimizatio
PNP/FiniteConsistencyBound.lean	finite uniform consistency
PNP/FiniteUniformRecovery.lean	threshold corollaries guar
PNP/BitFamilyUniformRecovery.lean	bit-budget exact-recovery
PNP/FiniteUniformRecoveryRegression.lean	regression wrappers for th
PNP/BitFamilyUniformRecoveryBinary.lean	two-point-domain speciali
PNP/BitFamilyUniformRecoveryBinaryRegression.lean	regression wrappers for th
PNP/FiniteUniformRate.lean	rational uniform-rate bou
PNP/FiniteUniformSuccess.lean	miss-ratio decay, non-dec
PNP/FiniteUniformSuccessRegression.lean	regression wrappers for th
PNP/BitFamilyUniformSuccessBinary.lean	Boolean 1/2-miss quantit
PNP/BitFamilyUniformSuccessBinaryRegression.lean	regression wrappers for B
PNP/ClockedKpolyBridge.lean	construction, family realiz
PNP/ClockedKpolyBridgeRegression.lean	regression wrappers for th
PNP/ClockedKpolyTargetInterface.lean	repackages exact bit-budg
PNP/ClockedKpolyTargetInterfaceRegression.lean	regression wrappers for cl
PNP/ClockedKpolyBridgeEquivalence.lean	clocked realization equiva
PNP/ClockedKpolyBridgeEquivalenceRegression.lean	regression wrappers for cl
PNP/ClockedKpolyGapAssessment.lean	bundles the clocked finite
PNP/ClockedKpolyGapAssessmentRegression.lean	regression wrappers for th
PNP/ClockedKpolyActualGapClosure.lean	closes the clocked payloa
PNP/ClockedKpolyActualGapClosureRegression.lean	regression wrappers for th
PNP/ActualSwitchedLocalFamily.lean	manuscript-shaped local f
PNP/ActualSwitchedLocalFamilyRegression.lean	regression wrappers for th
PNP/ActualSwitchedLocalStructure.lean	bounded-code and shared
PNP/ActualSwitchedLocalStructureRegression.lean	regression wrappers for th
PNP/ActualSwitchedLocalERMConstruction.lean	concrete actual switched-
PNP/ActualSwitchedLocalERMConstructionRegression.lean	regression wrappers for th
PNP/ActualSwitchedLocalCodeConstruction.lean	concrete actual switched-
PNP/ActualSwitchedLocalCodeConstructionRegression.lean	regression wrappers for th
PNP/ActualSwitchedLocalPluginLookupObstruction.lean	manuscript plug-in looku
PNP/ActualSwitchedLocalPluginLookupObstructionRegression.lean	regression wrappers for th
PNP/ActualSwitchedLocalPluginMajorityObstruction.lean	counted plug-in majority
PNP/ActualSwitchedLocalPluginMajorityObstructionRegression.lean	regression wrappers for th
PNP/ActualSwitchedLocalPluginSampleMajorityObstruction.lean	finite-sample plug-in majo
PNP/ActualSwitchedLocalPluginSampleMajorityObstructionRegression.lean	regression wrappers for th
PNP/ActualSwitchedLocalBoundedSampleMajorityObstruction.lean	bounded finite-sample plu
PNP/ActualSwitchedLocalBoundedSampleMajorityObstructionRegression.lean	regression wrappers for th
PNP/ClockedKpolyCompressionObstruction.lean	surjectivity lower bound f
PNP/ClockedKpolyCompressionObstructionRegression.lean	regression wrappers for cl
PNP/FiniteIIDAgreement.lean	exact weighted one-predic
PNP/FiniteIIDFamilyBound.lean	weighted deceptive-mass
PNP/FiniteIIDRecovery.lean	weighted exact-ERM recc

The point of the table is simple: the grassroots route is not starting from zero. We already have a serious computability substrate, an observer-relative semantics layer, and a new optimistic interface for small hypothesis classes.

3 Finite-Count Conditional Independence

The grassroots route also needs a probability-free language for conditioning and decorrelation. This matters because several manuscript-style repairs move between exact source events, randomized recodings, and output observations. Before any asymptotic claim is meaningful, the finite counting statement must be unambiguous.

The base exact definition is `CountIndependentWithin` from `ConditioningObstruction.lean`. It says that events E and F are conditionally independent inside a finite conditioning event C by the division-free equation

$$|C|\,|C \cap E \cap F| = |C \cap E|\,|C \cap F|.$$

The approximate layer in `ApproximateDecorrelationObstruction.lean` orients the two possible failures of this equation as a forward and backward natural-number gap. The new grassroots module `FiniteCountIndependenceFoundation.lean` packages those two gaps into a single symmetric defect:

$$\texttt{countIndependentWithinDefect} = \max(\texttt{forwardGap}, \texttt{backwardGap}).$$

Theorem 3.1 (Tolerance is max-defect boundedness)**.** *For finite events C, E, F,*

$$\textit{CountIndependentWithinTolerance}\ C\ E\ F\ \tau \iff \textit{countIndependentWithinDefect}\ C\ E\ F \le \tau.$$

This is `countIndependentWithinTolerance_iff_defect_le`. The same module proves monotonicity in the tolerance and the exact bridge

$$\texttt{CountIndependentWithinTolerance}\ C\ E\ F\ 0 \iff \texttt{CountIndependentWithin}\ C\ E\ F,$$

formalized as `countIndependentWithinTolerance_zero_iff_countIndependentWithin`. Equivalently,

$$\texttt{countIndependentWithinDefect}\ C\ E\ F = 0$$

is exactly the original cross-multiplied finite-count independence equation.

The module also lifts this to output variables. A certificate `CountIndependentWithinToleranceOutput` specializes to every output fiber, with direct projection lemmas for the forward and backward defects. At zero tolerance it is exactly exact finite-count independence for every output fiber:

`countIndependentWithinToleranceOutput_zero_iff_countIndependentWithin_fibers`.

For a finite output type, the file defines `countIndependentWithinOutputDefect`, the maximum fiber defect over all outputs, and proves

$$\texttt{CountIndependentWithinToleranceOutput}\ C\ E\ Y\ \tau \iff \texttt{countIndependentWithinOutputDefect}\ C\ E\ Y \le$$

This is intentionally small infrastructure, but it changes the shape of later work. Future positive routes can target a clean finite-count certificate; crux results can be read as applications showing that certain recoding interfaces force that certificate to have a large defect.

4 Finite-Coin Source Erasure

The finite-count layer above deliberately treats decorrelation as a property of events and output fibers. A separate grassroots question is what an output recoding can still preserve once every finite-coin output fiber is balanced between the true and false source sides. The answer is now formalized in `FiniteCoinSourceErasureFoundation.lean`.

For a recoding $r : \text{Bool} \to \text{Coin} \to \alpha$, the file defines `finiteCoinOutputPredicateNeutral`: a decidable output predicate $P : \alpha \to \text{Prop}$ is neutral when it selects the same number of true-side and false-side coin samples. It then defines `FiniteCoinOutputPredicateErasing`r to mean that every decidable output predicate is neutral.

Theorem 4.1 (Balanced fibers are predicate erasure)**.** *If α has decidable equality, then*

$$\textit{FiniteCoinRecodingFiberBalanced}\ r \quad \Longleftrightarrow \quad \textit{FiniteCoinOutputPredicateErasing}\ r.$$

This is `finiteCoinRecodingFiberBalanced_iff_outputPredicateErasing`. The forward direction says balanced fibers erase not only singleton outputs but every decidable observation of the output. The reverse direction uses singleton predicates, so the equivalence is exact rather than an artifact of one chosen statistic.

The same file records strict refuters. Any output predicate with a strict true-side or false-side count advantage contradicts predicate erasure. If the coin space is nonempty, a predicate that uniformly separates the two source sides also contradicts predicate erasure. A structural specialization is now included as `FiniteCoinSourceRangesSeparated`: disjoint true-side and false-side output ranges refute predicate erasure, and with decidable output equality they also construct an explicit Boolean source decoder by testing membership in the true-side output range.

Finally, the file packages the decoder consequence. For a Boolean decoder $\text{decode} : \alpha \to \text{Bool}$, it defines the aggregate correct count

$$\#\{c : \text{Coin} \mid \text{decode}(r(\text{true}, c)) = \text{true}\} + \#\{c : \text{Coin} \mid \text{decode}(r(\text{false}, c)) = \text{false}\}.$$

Predicate erasure forces this count to be exactly $|\text{Coin}|$, i.e. half of the two-sided sample space in division-free form. Therefore any decoder with strictly better-than-half aggregate recovery, or any decoder that uniformly recovers the source bit on a nonempty coin space, refutes erasure.

Equivalently, the file defines `FiniteCoinOutputSourceRecovering` for the existence of such a decoder and proves that source-recovering outputs are incompatible with predicate erasure. This keeps the future positive route honest: preserving the source bit at the output level is a named mathematical property, not a prose-side intuition.

This is a grassroots strengthening of the balanced-fiber crux. It does not merely say that the manuscript's current output statistic fails. It proves that the whole finite-coin balanced-fiber regime is source-erasing at the level of all decidable output predicates and Boolean decoders.

4.1 Source Erasure as Independence

The source-erasure layer is now tied back to the finite-count independence API in `FiniteCoinIndependenceBridge.l`. The file works on the concrete two-sided sample space $\text{Bool} \times \text{Coin}$. It defines `finiteCoinSourceTrue` for the source-bit event and `finiteCoinOutput` for the output variable induced by a finite-coin recoding.

The first part of the bridge is counting infrastructure:

$$|\{(b, c) : b = \text{true}\}| = |\text{Coin}|,$$

and for every decidable output predicate P,

$$|\{(b,c) : P(r(b,c))\}| = |\{c : P(r(\text{true},c))\}| + |\{c : P(r(\text{false},c))\}|.$$

These identities support the exact theorem

`finiteCoinOutputPredicateNeutral` $r\ P \iff$ `CountIndependentWithin` $\top$ `finiteCoinSourceTrue` ($P \circ$ `fin`

This is formalized as `finiteCoinOutputPredicateNeutral_iff_countIndependentWithin_trueSource_outputP` Consequently, predicate erasure is exactly independence from the source bit for every decidable output predicate.

For output variables with decidable equality, the bridge specializes this to the fiberwise API:

`FiniteCoinRecodingFiberBalanced` $r \iff$ `CountIndependentWithinToleranceOutput` $\top$ `finiteCoinSour`

For finite output types, it also proves the max-defect form:

`FiniteCoinRecodingFiberBalanced` $r \iff$ `countIndependentWithinOutputDefect` $\top$ `finiteCoinSourceT`

This matters for the ground-up route because the balanced-fiber branch is no longer merely a named escape hatch. It is exactly the zero-defect source/output independence condition in the same finite-count language used by the crux obstructions.

4.2 Defect Equals Scaled Fiber Imbalance

The next quantitative step is in `FiniteCoinDefectFormula.lean`. For an output fiber y, define its finite-coin imbalance as

$$\max(T_y - F_y,\ F_y - T_y),$$

where T_y and F_y are the true-side and false-side coin-fiber counts. The file proves that zero imbalance is exactly $T_y = F_y$.

More importantly, it computes the oriented finite-count defects on Bool $\times$ Coin:

$$\texttt{forwardGap}(y) = |\text{Coin}|(T_y - F_y), \qquad \texttt{backwardGap}(y) = |\text{Coin}|(F_y - T_y).$$

Therefore the symmetric fiber defect is exactly

$$|\text{Coin}| \cdot \max(T_y - F_y,\ F_y - T_y).$$

The approximate output certificate has an exact reformulation:

`CountIndependentWithinToleranceOutput` $\top$ `finiteCoinSourceTrue` (`finiteCoinOutput` r) $\tau \iff \forall y,\ |\text{Co}$

For finite output types, `countIndependentWithinOutputDefect` is the finite supremum of these scaled fiber imbalances.

This turns the balanced-fiber story into a quantitative foundation: exact balance is zero scaled imbalance, and approximate decorrelation is no stronger or weaker than bounding every scaled fiber imbalance.

4.3 Tolerance Quantization

`FiniteCoinToleranceQuantization.lean` records the immediate but useful threshold consequence of the scaled-defect formula. If a finite-coin fiber imbalance is nonzero, then its scaled contribution is at least one whole |Coin| block:

$$\texttt{fiberImbalance}(r, y) \neq 0 \quad \Longrightarrow \quad |\text{Coin}| \leq |\text{Coin}| \cdot \texttt{fiberImbalance}(r, y).$$

This is `finiteCoin_card_le_scaled_fiberImbalance_of_ne_zero`.

Consequently, any scaled fiber-imbalance bound below |Coin| forces that fiber to be exactly balanced. At the output-variable level, the file proves

$\tau < |\text{Coin}| \quad \Longrightarrow \quad$ (`CountIndependentWithinToleranceOutput` $\top$ `finiteCoinSourceTrue` (`finiteCoinOutput`

This is `countIndependentWithinToleranceOutput_finiteCoinOutput_lt_card_iff_fiberBalanced`.

For finite output types, the max-defect form is also pinned down. If the maximum output defect is below |Coin|, the recoding has balanced fibers; conversely, every unbalanced finite-output recoding has output defect at least |Coin|. When the coin space is nonempty, defect below one coin block is equivalent to balanced fibers:

`countIndependentWithinOutputDefect` $\top$ `finiteCoinSourceTrue` (`finiteCoinOutput` r) $< |\text{Coin}| \quad \Longleftrightarrow \quad$ `Fin`

This gives the finite-coin grassroots layer a discrete threshold theorem: "tiny" approximate source/output independence, below one coin block of finite-count defect, is not a genuine relaxation of balanced-fiber erasure.

4.4 Deterministic Postprocessing

`FiniteCoinPostprocessingFoundation.lean` adds the postprocessing laws needed to keep the finite-coin layer stable under ordinary recodings of the output. If $g : \alpha \to \beta$ is a deterministic map, exact predicate erasure is preserved:

$$\texttt{FiniteCoinOutputPredicateErasing}\ r \quad \Longrightarrow \quad \texttt{FiniteCoinOutputPredicateErasing}\ (g \circ r).$$

With decidable equality on the source and postprocessed output types, this gives the balanced-fiber form:

$$\texttt{FiniteCoinRecodingFiberBalanced}\ r \quad \Longrightarrow \quad \texttt{FiniteCoinRecodingFiberBalanced}\ (g \circ r).$$

The approximate statement is deliberately quantitative. For finite original output type α, the file defines `finiteOutputPreimageFiber`(g, z) and `finiteOutputMapFiberCardBound`(g, m), meaning every postprocessed fiber merges at most m original output values. It proves the count identities

$$T'_z = \sum_{y : g(y) = z} T_y, \qquad F'_z = \sum_{y : g(y) = z} F_y,$$

and the imbalance inequality

$$\texttt{fiberImbalance}(g \circ r, z) \leq \sum_{y : g(y) = z} \texttt{fiberImbalance}(r, y).$$

Consequently, if r has finite-coin output tolerance τ, then

$$\texttt{CountIndependentWithinToleranceOutput}(r, \tau) \quad \Longrightarrow \quad \texttt{CountIndependentWithinToleranceOutput}(g \circ r, m\tau).$$

This is `countIndependentWithinToleranceOutput_finiteCoinOutput_postcompose`. For finite postprocessed output types, the max-defect form is:

$$\texttt{outputDefect}(g \circ r) \leq m \, \texttt{outputDefect}(r).$$

The file also records the trivial bound $m = |\alpha|$, so every finite postprocessing has an explicit worst-case blowup.

This blocks another common ambiguity in repair attempts. Deterministic postprocessing cannot restore source information after exact erasure; for approximate certificates it can only aggregate already present fiber defects, with the aggregation controlled by finite preimage size.

4.5 Route-Level Scoping

The finite-coin stack is useful, but it is not a stop-grade route closure by itself. The crux synthesis ledger now makes this explicit in `CruxSynthesis.lean`. The covered finite-coin subrepair classes are: same-source finite-coin marginal recoding, predicate erasure, decoder half-accuracy, exact scaled fiber imbalance, sub-cardinality tolerance quantization, deterministic exact postprocessing, and finite-preimage approximate postprocessing with $\tau \mapsto m\tau$ blowup. The same synthesis ledger now also tracks a narrow finite-coin randomized-residual subledger: supportwise unresolved product-space residuals have no positive advantage, and any positive finite-coin randomized-residual advantage must produce a positive-weight resolving source/coin pair. On the exact post-switch surface this becomes a positive-weight input/coin pair that distinguishes the local-input switch.

The same ledger explicitly records what this does *not* cover. The broad `randomizedResidual`, `residualSideInformation`, `approximateDecorrelation`, and `kpolyCompressionBridge` repair classes remain outside the current packet. Thus the finite-coin work should be read as a precise foundation for a same-source finite-count subroute and as route-level guidance for future attacks, not as a proof that all randomized or approximate repairs are blocked.

Because the finite-coin subledger is now explicit, the synthesis ledger selects `kpolyCompressionBridge` as the next highest-marginal target. This is the remaining manuscript-facing bridge after the abstract clocked finite-uniform counting kernel has been separated and covered. `ClockedKpolyBridgeEquivalence.lean` now tightens that target further: existence of a clocked s-bit realization family is equivalent to the existing exact visible compression target, and changing the external clock bound does not change realizability. So the next target is a real small-class theorem for the switched family, not a clock-bookkeeping repair. The constructive direction is now explicit rather than hidden inside the existential proof. `ClockedKpolyBridge.lean` defines `ClockedBitCodeFamily.ofBitEncodedClassifierFamily`, which equips an ordinary s-bit classifier family with an external clock without changing its decoder, realization predicate, exact-recovery rate, or non-deceptive sample rate. The target interface uses this constructor in `IndexedPredictorFamily.clockedBitCodeFamilyOfHasBitBudget`; Lean now also proves that the underlying bit family of this constructed clocked family realizes the original indexed family and re-witnesses the same bit budget. Finally, `IndexedPredictorFamily.realizedByClockedBitFamily_cl` and `realizedByClockedExactVisibleFamily_of_exactVisibleCompressionTarget` state the construction as an actual family-level clocked realization theorem. Thus the bridge no longer depends on a nameless clocked witness: given a bit-budget theorem, the clocked family is canonically constructed. The family-level predicate `IndexedPredictorFamily.RealizedByClockedBitFamily` and its forgetful theorem `IndexedPredictorFamily.hasBitBudget_of_realizedByClockedBitFamily` now live in `ClockedKpolyBridge.lean` itself. Consequently `ClockedKpolyBridgeEquivalence.lean` no longer imports the obstruction module; the obstruction file consumes the core realization interface rather than defining it.

The bridge now carries the full finite-success rate pair rather than only the realized-target half. `ClockedKpolyBridge.lean` defines `ClockedBitCodeFamily.nondeceptiveRate` and proves the same $1-2^s\rho^m$ lower bound and strict positivity statements that the underlying bit-family layer already had. The target interface then lifts these to any exact s-bit budget and to any exact visible compression target via `IndexedPredictorFamily.nondeceptiveRate_ge_one_sub_uniformMissRatio_pow_of_hasBitBu` and `nondeceptiveRate_ge_one_sub_uniformMissRatio_pow_of_exactVisibleCompressionTarget`. This matters because the manuscript-facing bridge now exposes both target-independent non-deceptive samples and exact recovery for realized targets from the same finite-count hypothesis.

The bridge also lifts the pure finite-count existence layer. Below the bit-budget threshold, `ClockedBitCodeFamily.exists_nondeceptiveSample_of_bitBudget_bound_lt` produces an explicit non-deceptive point sample for the underlying clocked family, and `ClockedBitCodeFamily.exists_sample_em` produces a point sample on which ERM exactly recovers any realized target. The target interface lifts both statements to exact `HasBitBudget` witnesses and to exact visible compression targets. Thus the clocked bridge now exposes both quantitative rates and concrete sample witnesses, while still leaving the small-image theorem for the manuscript's switched family as the real burden.

`ClockedKpolyGapAssessment.lean` makes this assessment explicit. It defines `ClockedKpolyFiniteLearningPa` bundling a clocked exact visible realization family with the threshold good-sample and exact ERM recovery witnesses for every selected predictor. Lean proves

`ClockedKpolyFiniteLearningPayload` $G\ s$ `clock` $\iff$ `ExactVisibleCompressionTarget` $G\ s$ $\iff$ `Fini`

It also lifts the existing surjectivity and injective finite-probe lower bounds to this full payload. So the accumulated clocked bridge is locally complete: all finite-learning consequences now follow from the exact visible compression target. But the route is not closed, because the target itself is still the unproved small finite-image theorem for the actual switched family.

`ClockedKpolyActualGapClosure.lean` resolves the next fork for the formalized exact ZAB ERM route. For the concrete ERM-selected family `exactZABDecisionListERMFamily`, Lean proves the clocked finite-learning payload at budget $r+2k+1$: `exactZABDecisionListERMClockedKpolyFiniteLearningPayl` The bit-valued manuscript-facing specialization is `bitVecZABDecisionListERMClockedKpolyFiniteLearningPayl` For an arbitrary family variable G, the minimal transfer assumption is the exact identification

$$G = \texttt{bitVecZABDecisionListERMFamily samples},$$

captured by `clockedKpolyFiniteLearningPayload_of_eq_bitVecZABDecisionListERMFamily`; the packaged recovery-data interface also closes the same payload via `BitVecZABERMRecoveryData.clockedKpolyFi` Without such an exact-family identification, the present interface remains insufficient: the full exact-visible Boolean family is still a legal `ExactVisibleSwitchedFamily`, and `currentExactVisibleInterface_not_fo` refutes any theorem deriving the clocked payload for every exact-visible family below the full visible truth-table budget. The synthesis ledger now absorbs this exact fork in `currentPNPStrongestHonestStatus`: the exact ZAB closure is recorded as narrow *Kpoly* subrepair progress, while the broad manuscript-specific `kpolyCompressionBridge` remains explicitly uncovered.

`ActualSwitchedLocalFamily.lean` moves the same fork one step closer to the manuscript's actual local normal form. It defines `ActualSwitchedLocalInterface` with fields

$$z : \Phi \to Z, \qquad a : I \to \Phi \to \{0,1\}^k, \qquad b : \Phi \to \{0,1\}^k,$$

a local rule $h_i : (z, a, b) \to \text{Bool}$, and the equation

$$output_i(\phi) = h_i(z(\phi), a_i(\phi), b(\phi)).$$

For this actual-local interface, Lean proves the same conditional closure: if the induced exact-visible predictor family is exactly `bitVecZABDecisionListERMFamily samples`, then `ClockedKpolyFiniteLearningPayl` holds at budget $r + 2k + 1$. But Lean also proves the locality-only obstruction. Taking blocks themselves to be exact visible inputs and indices to be all Boolean rules gives a legal actual-local interface whose predictor family is the full exact-visible Boolean family. Hence, below the full visible truth-table budget, `actualSwitchedLocalInterface_not_forall_clockedKpolyFiniteLearningPayload_of_lt_surfaceC` refutes any derivation of the clocked payload from the current actual-local normal-form assumptions alone.

`ActualSwitchedLocalStructure.lean` records the next narrowed proof obligation. The extra assumption is not more locality; it is that the local rules selected by the actual switched family come from a bounded code class. The structure `ActualSwitchedLocalInterface.BitCodeData` supplies one s-bit encoded family and proves

$$\texttt{ExactVisibleCompressionTarget}\ T.\texttt{predictorFamily}\ s, \qquad T.\texttt{predictorFamily.FinitePredictorCover}(2^s),$$

and therefore `ClockedKpolyFiniteLearningPayload` at the same budget. This is the weakest formal closure currently isolated: with a finite local code/quotient map, the small-image gap closes. The same file proves that this condition is substantive by refuting it for `fullRuleActualSwitchedLocalInterface` whenever s is below the exact visible truth-table budget.

The file also gives a concrete manuscript-shaped sufficient instance: `ActualSwitchedLocalInterface.ZABDecis` It requires the actual local rules to be realized by the existing shared $(zfeat(z), a, b)$ decision-list family, without requiring equality to a particular ERM selector. Lean then proves a finite predictor cover, finite representative-index cover, finite predictor quotient, and the clocked payload at budget $r + 2k + 1$. Under

$$r + 2k + 1 < |\texttt{ExactVisiblePostSwitchSurface}\ Z\ k|,$$

Lean also proves that no such ZAB-structured family can have a surjective full Boolean predictor image, so the prior full-rule local counterexample is excluded. Thus the current exact status is a minimal missing-assumption result: locality-only is insufficient; bounded local code data closes the finite-image target; and shared ZAB decision-list realization is one concrete candidate for the manuscript-derived non-arbitrary structure that would supply that code data. The exact ERM equality hypothesis now factors through this narrowed interface: `zabDecisionListData_of_eq_exactZABDecisionListERMFamily` constructs `ZABDecisionListData` from equality to the concrete shared ERM family, and `bitCodeData_of_eq_exact` constructs the bounded-code assumption itself. The bit-valued identity-extractor version has matching constructors for `bitVecZABDecisionListERMFamily`. Thus exact ERM equality is now a sufficient way to discharge the minimal finite-image assumption, not a separate parallel shortcut.

`ActualSwitchedLocalERMConstruction.lean` removes one layer of that conditionality for the ERM-shaped construction itself. Given concrete manuscript-visible maps

$$zOf : \Phi \to Z, \qquad aOf : I \to \Phi \to \{0,1\}^k, \qquad bOf : \Phi \to \{0,1\}^k,$$

an extractor $zfeat : Z \to \{0,1\}^r$, and indexed samples, Lean constructs

$$\texttt{exactZABDecisionListERMConstruction}\ zOf\ aOf\ bOf\ zfeat\ samples : \texttt{ActualSwitchedLocalInterface}.$$

Its predictor family is definitionally $\texttt{exactZABDecisionListERMFamily}(zfeat, samples)$. Consequently Lean proves, with no additional assumption, that this constructed actual-local family satisfies `ActualSwitchedLocalInterface.ZABDecisionListData` and `ActualSwitchedLocalInterface.BitCodeData` at budget $r + 2k + 1$, and then proves

$$\texttt{FinitePredictorCover}\ (2^{r+2k+1}) \quad \text{and} \quad \texttt{ClockedKpolyFiniteLearningPayload}\ (r{+}2k{+}1)\ clock.$$

The bit-valued identity-extractor specialization is formalized as `bitVecZABDecisionListERMConstruction`. This is now a concrete construction theorem, not merely a strengthened-interface hypothesis. The remaining manuscript-specific obligation is exact and narrower: prove that the paper's switched family is this ERM construction, or else supply the same per-index ZAB decision-list codes through `zabDecisionListDataOfCodes`. Without that identification, the earlier locality-only full-rule counterexample still shows that actual-local shape alone cannot imply a small predictor image.

`ActualSwitchedLocalCodeConstruction.lean` formalizes that second route directly. Given the same visible maps zOf, aOf, bOf, extractor $zfeat$, and a function

$$codes : I \to \texttt{SharedAffineDecisionListCode}\,(r + (k + k)),$$

Lean constructs `exactZABCodeConstruction`. Its exact-visible predictor family is definitionally the canonical ZAB code family induced by those codes, and the actual output is the corresponding decision-list prediction evaluated at $(zOf(\phi), aOf_i(\phi), bOf(\phi))$. From this concrete construction Lean proves `exactZABCodeConstruction_zabDecisionListData`, `exactZABCodeConstruction_bitCodeData`, `exactZABCodeConstruction_finitePredictorCover`, and `exactZABCodeConstruction_clockedKpolyFiniteLe`. Thus an explicit per-index ZAB code assignment is now enough to close the exact small-image target for an actual switched-local wrapper, independently of the ERM selector. The manuscript burden is correspondingly reduced to either an ERM equality proof or a code extraction proof for its local rules.

`ActualSwitchedLocalPluginLookupObstruction.lean` checks the literal finite-alphabet plug-in endpoint described in the v11/arXiv manuscript paragraph: the local rule is implemented by a hash-table lookup on the exact post-switch alphabet $u = (z, a_i, b)$, with no hypothesis enumeration. Lean formalizes that unrestricted endpoint as `pluginLookupActualSwitchedLocalInterface`. Its predictor family is the full exact-visible Boolean rule family, and `pluginLookupActualSwitchedLocalInterface_surject` proves that every Boolean lookup table on the post-switch alphabet is realized. Consequently, below the exact visible truth-table budget, Lean proves

`pluginLookupActualSwitchedLocalInterface_not_finitePredictorCover_of_lt_surfaceCard`,

the corresponding clocked-payload obstruction, and the two constructor obstructions `pluginLookupActualSwitched` and `pluginLookupActualSwitchedLocalInterface_not_zabDecisionListData_of_lt_surfaceCard`. Thus a finite-alphabet lookup implementation is not itself the missing small-image theorem. It closes only after an additional manuscript theorem identifies those lookup tables with the ZAB ERM family or with explicit per-index ZAB codes.

`ActualSwitchedLocalPluginMajorityObstruction.lean` removes a possible escape hatch in that diagnosis. The manuscript states the local rule as a plug-in majority over training labels grouped by the visible input u. Lean formalizes the aggregated version as `PluginMajorityCounts`: for each visible input, record the number of true and false votes and return the majority label. The theorem `pluginMajorityActualSwitchedLocalInterface_realizes_every_lookupTable` proves that every Boolean lookup table is still realized, by placing one vote on the desired side for each visible input. Therefore `pluginMajorityActualSwitchedLocalInterface_surjective` and the matching no-cover/no-payload/no-`BitCodeData`/no-`ZABDecisionListData` theorems show that the counted majority endpoint is also full-rule expressive below the exact visible truth-table budget. The obstruction is therefore not caused by replacing majority with a lookup table; it is caused by the absence of a proved restriction from the plug-in majority counts to the small ZAB/code class.

`ActualSwitchedLocalPluginSampleMajorityObstruction.lean` removes the last aggregation-only caveat from the plug-in majority blocker. Instead of postulating the true/false vote counts directly, Lean defines `PluginSampleMajority.trueVotes` and `PluginSampleMajority.falseVotes`

by counting labels in an actual finite sample. For every exact-visible Boolean rule $rule$, the graph sample

$$\{(u, rule(u)) : u \in \texttt{ExactVisiblePostSwitchSurface}\, Z\, k\}$$

has one labeled example at each visible input. Lean proves `PluginSampleMajority.predict_graphSample`, and then `pluginSampleMajorityActualSwitchedLocalInterface_surjective`: the sample-level plug-in majority interface is still surjective onto all Boolean rules on $u = (z, a, b)$. The companion no-cover/no-payload/no-`BitCodeData`/ no-`ZABDecisionListData` theorems therefore show that finite samples and empirical majority alone do not imply the manuscript's needed small predictor image. A successful route must still prove that the actual samples/rules are restricted to the existing exact ZAB ERM family or to explicit bounded ZAB codes.

`ActualSwitchedLocalBoundedSampleMajorityObstruction.lean` checks the next natural repair: bounding the finite sample length. Lean proves `PluginSampleMajority.length_graphSample`, namely that the graph sample of a rule has length exactly the cardinality of the visible alphabet. It then defines a bounded-sample plug-in majority interface whose indices are samples of length at most B. If

$$|\texttt{ExactVisiblePostSwitchSurface}\, Z\, k| \le B,$$

Lean proves `boundedSamplePluginMajorityActualSwitchedLocalInterface_surjective`. The strict opposite case is now also machine checked. Lean proves `boundedSamplePluginMajorityActualSwitchedLoca` if $B < |\texttt{ExactVisiblePostSwitchSurface}\, Z\, k|$, every bounded sample omits some visible input, where the current tie-breaking plug-in majority predicts `true`; hence the all-false rule is not realizable. Combining the two directions gives `boundedSamplePluginMajorityActualSwitchedLocalInterface_surjective` Consequently, sample length alone has an exact full-rule expressivity threshold at the visible alphabet size. Above that threshold the endpoint still has no finite predictor cover, clocked payload, `BitCodeData`, or `ZABDecisionListData` below the exact visible truth-table budget. Below that threshold Lean proves only non-surjectivity; a separate image-size, quotient, ZAB ERM, or code theorem is still needed to close the manuscript's small-image obligation.

`ExactVisibleImageBudget.lean` refines the same target in grassroots terms. Lean proves that an exact s-bit budget is equivalent to a finite cover of the predictor image by at most 2^s Boolean functions, and proves the same equivalence for the clocked exact-visible interface. So the remaining `kpolyCompressionBridge` burden can now be phrased without implementation noise: prove that the actual switched family has a finite predictor image of the required size. The theorem does not assert that this image is small; it identifies the exact finite-image obligation any successful repair must meet. The same module also proves that a finite predictor cover is equivalent to a finite representative-index cover: finitely many actual selected indices whose predictors represent every predictor in the family. This avoids treating the cover as an arbitrary external set of Boolean functions. It also proves the equivalent finite quotient-code presentation: every index is encoded into one of finitely many code values, and the decoded Boolean predictor is exactly the selected predictor for that index. The clocked exact-visible interface now has matching negative forms too: nonexistence of a clocked realization is equivalent to nonexistence of each of these finite-image presentations at the same 2^s budget. It now includes the transport layer needed for later route-building: predictor-image covers, representative-index covers, and quotient-code presentations are monotone in the allowed budget, pull back along an explicit factorization through a visible summary, and push back to that summary when the view has a section. This makes the finite-image obligation stable under the same kind of exact pullback arguments used elsewhere in the PNP development, without claiming that the actual switched family has a small image. The reduced-view lower bound is now also machine checked. If an exact family factors through a finite visible summary with a section, and the reduced family is surjective onto all Boolean classifiers on that summary, then every finite predictor-image cover,

finite representative-index cover, and finite quotient-code presentation of the exact family must have size at least $2^{|\text{summary}|}$. The raw (a, b)-surface specialization gives the concrete threshold $2^{2^{2k}}$ for `ABVisibleSurface` k. Thus finite-image compression after quotienting to (a, b) still requires an actual strict-subclass theorem for the reduced switched predictors. The same file now proves the matching finite-image counting obstruction: if an indexed family on a finite surface is still surjective onto all Boolean classifiers, then every finite predictor cover, finite representative-index cover, and finite quotient-code presentation has size at least the full classifier-space cardinality $2^{|\text{surface}|}$. In the exact visible clocked interface, this gives a direct no-go theorem below that predictor-card threshold. `ProductSuccessKpolyBridgeObstruction.lean` adds the corresponding route-level sanity check. A finite tower product bound, or even a valid fixed-field v13 switching certificate, is not a finite-image theorem for the exact visible switched family. Lean refutes product-only and fielded-only compression bridges using the full exact-visible Boolean family, including a nonempty one-step Boolean-square certificate with the one-cell field. So the grassroots *Kpoly* target remains a semantic predictor-image theorem, not a consequence of product success alone. The same module now pins the positive boundary too. Once the product premise or the fielded switching certificate is actually true, an implication from that premise to exact-visible compression is equivalent to a finite predictor-image cover of size at most 2^s. The global product-only and fielded-only bridge schemas are therefore exactly schemas demanding such covers for every exact-visible family; when instantiated at one family, they also yield finite representative-index covers and quotient-code presentations. This keeps the next grassroots obligation concrete: build the finite image of the actual switched predictors, or exhibit a fully expressive subcase that blocks it. `CruxSynthesis.lean` now records this status in the narrow *Kpoly* subrepair ledger: product-bound and fielded-switching bridge schemas are covered only as finite-image-boundary reductions and full-visible refuters. The broad manuscript-facing *Kpoly* bridge remains a live gap until the actual switched predictor class is proved genuinely small. The route check is no longer tied to the canonical full-rule family: any exact-visible family with a surjective prediction map now blocks the product-only or fielded-only bridge below the same surface threshold, and blocks the semantic finite-image bridge below the corresponding predictor-cardinality threshold. So changing the index presentation of a fully expressive family is not a repair.

5 Encoded Hypothesis Families

The key new grassroots abstraction is that a hypothesis class should be represented by a finite code space together with a decoder.

> code budget first, realized class second.

Formally, `HypothesisClass.lean` introduces an *encoded family*

$$\text{decode} : \text{Code} \to (\text{Input} \to \text{Output}),$$

where Code is finite. The realized hypothesis class is the range of decode.

Theorem 5.1 (Realized class is bounded by code space)**.** *For any encoded family H,*

$$|\text{realized}(H)| \le |\text{Code}(H)|.$$

This is the theorem `EncodedFamily.card_realized_le` in Lean.

The specialization relevant to the present route is the bit-coded case.

Theorem 5.2 (Bit-budget bound)**.** *If a family of Boolean classifiers is decoded from s bits, then it realizes at most 2^s distinct classifiers.*

This is formalized as `BitEncodedClassifierFamily.card_realized_le`.

This theorem is elementary, but it captures the exact shape of the optimistic repair that would be needed later. A switching theorem cannot stop at "the new predictor is local." It must continue to a statement of the form:

> the switched predictors lie in a family decoded from polylog(m) bits.

Only then does finite-class learning theory become genuinely available.

6 Finite-Class ERM Kernel

The next honest step after a finite encoded family is not yet a PAC theorem. It is the finite combinatorial kernel of empirical risk minimization.

The new module `FiniteERM.lean` defines labeled samples as finite lists of pairs (x, y), empirical error as the number of misclassified examples in the sample, and exact sample consistency as zero empirical error.

Theorem 6.1 (Finite ERM existence)**.** *Let H be a nonempty encoded family. For every finite labeled sample S, there exists a realized predictor $h^\star \in \mathrm{realized}(H)$ such that*

$$\mathrm{err}_S(h^\star) \le \mathrm{err}_S(g) \qquad \textit{for all } g \in \mathrm{realized}(H).$$

This is formalized as `EncodedFamily.exists_realized_empiricalRiskMinimizer`.

Theorem 6.2 (Zero-error transfer through ERM)**.** *If some code in the family fits the sample exactly, then the chosen ERM predictor also fits the sample exactly.*

This is formalized as `EncodedFamily.empiricalRiskPredictor_fitsSample_of_exists_code_fits`.

This is still modest, but it matters. Combined with the code-budget theorem from `HypothesisClass.lean`, it gives the first clean bridge from "small encoded family" to an actual learning-theoretic construction.

7 Finite Uniform Consistency Bound

The next step now formalized is a fully combinatorial finite-domain bound. We do not yet assume an arbitrary distribution or invoke measure theory. Instead, we count how many length-m input samples can make a wrong predictor look perfectly consistent with the target labels.

Theorem 7.1 (Wrong predictors fit only exponentially few samples)**.** *Let X be a finite input domain. If $h \neq f$, then the number of length-m samples on which h agrees with the target f at every sampled point is at most*

$$(|X| - 1)^m.$$

This is formalized as `card_consistentSamples_le_of_exists_disagreement` in `FiniteConsistencyBound.lean`.

Theorem 7.2 (Finite encoded families have few deceptive samples)**.** *Let H be an encoded family over a finite input domain X. The number of length-m samples on which some wrong code in H still matches all target labels is at most*

$$|\mathrm{Code}(H)| \cdot (|X| - 1)^m.$$

This is formalized as `EncodedFamily.card_deceptiveSamples_le`.

Theorem 7.3 (ERM recovers the target off the deceptive set)**.** *If the target predictor is realized by the encoded family, then on every non-deceptive sample the chosen ERM predictor equals the target predictor exactly.*

This is formalized as `EncodedFamily.empiricalRiskPredictor_eq_target_of_not_deceptive`.

This is not yet the full finite-class generalization theorem one would state in terms of $\log|H|$ and an arbitrary data distribution. But it is already the right union-bound skeleton in a completely concrete finite setting.

8 Finite Uniform Recovery Threshold

The next corollary is the cleanest possible extraction from the deceptive-sample bound. If the deceptive set is strictly smaller than the whole length-m sample space, then at least one non-deceptive sample exists.

Theorem 8.1 (Threshold for existence of a good sample)**.** *Let H be an encoded family over a finite input domain X. If*

$$|\mathrm{Code}(H)| \cdot (|X| - 1)^m < |X|^m,$$

then there exists a length-m sample outside the deceptive set.

This is formalized as `EncodedFamily.exists_nondeceptiveSample_of_bound_lt` in `FiniteUniformRecovery.lea`

Theorem 8.2 (Threshold corollary for exact ERM recovery)**.** *If the target predictor is realized by the encoded family and*

$$|\mathrm{Code}(H)| \cdot (|X| - 1)^m < |X|^m,$$

then there exists a length-m sample on which the chosen ERM predictor equals the target predictor exactly.

This is formalized as `EncodedFamily.exists_sample_empiricalRiskPredictor_eq_target_of_bound_lt`.

The same counting threshold now has an explicit bit-budget form. For an s-bit Boolean family F, the generic code-cardinality factor becomes 2^s. Thus

$$2^s(|X| - 1)^m < |X|^m$$

implies the existence of a non-deceptive sample for F. If the target is realized by F, the bit-family ERM wrapper exactly recovers the target on some labeled point sample. These statements are formalized as `BitEncodedClassifierFamily.exists_nondeceptiveSample_of_bitBudget_bound_lt` and `BitEncodedClassifierFamily.exists_sample_empiricalRiskPredictor_eq_target_of_bitBudget_bound_lt` with regression wrappers in `FiniteUniformRecoveryRegression.lean`.

There is also a two-point-domain specialization. If $|X| = 2$, the same threshold simplifies to

$$s < m.$$

Indeed $2^s(|X|-1)^m < |X|^m$ becomes $2^s < 2^m$. The module `BitFamilyUniformRecoveryBinary.lean` formalizes this as `BitEncodedClassifierFamily.bitBudget_bound_lt_of_card_eq_two_of_lt_sampleSize`, then derives non-deceptive-sample and exact ERM recovery corollaries for any finite two-point input domain, with Boolean-domain wrappers as the canonical binary case.

The clocked *Kpoly* bridge now carries the same existential layer. Theorems `ClockedBitCodeFamily.exists_non` and `ClockedBitCodeFamily.exists_sample_empiricalRiskPredictor_eq_target_of_bitBudget_bound_lt` lift the generic bit-budget threshold to clocked families. On the Boolean domain, `ClockedBitCodeFamily.exists_bo` and `ClockedBitCodeFamily.exists_bool_sample_empiricalRiskPredictor_eq_target_of_lt_sampleSize` give the concrete $s < m$ versions. These are constructive consequences of a small clocked bit family; they do not assert that the exact switched family has one.

This is still finite and non-asymptotic, but it clarifies the next job. The rate layer below turns the combinatorial ratio behind the threshold into an explicit uniform success bound; the later weighted layer relaxes the uniform sampling assumption.

9 Finite Uniform Rate Layer

The next natural refinement is still finite, but no longer purely existential. Instead of asking only whether some good sample exists, we measure how large the good and bad regions are inside the finite sample space.

The module `FiniteUniformRate.lean` defines rational uniform densities on the point-sample space:

1. the deceptive rate,
2. the non-deceptive rate,
3. the exact-recovery rate for ERM.

Theorem 9.1 (Uniform deceptive-rate bound)**.** *Let H be an encoded family over a nonempty finite input domain X. Then the uniform fraction of length-m samples that are deceptive is at most*

$$\frac{|\mathrm{Code}(H)| \cdot (|X| - 1)^m}{|X|^m}.$$

This is formalized as `EncodedFamily.deceptiveRate_le_bound` in `FiniteUniformRate.lean`.

Theorem 9.2 (Exact recovery dominates the non-deceptive region)**.** *If the target predictor is realized by the encoded family, then the uniform exact recovery rate of the chosen ERM predictor is at least*

$$1 - \mathrm{rate}_{\mathrm{deceptive}}.$$

This is formalized as `EncodedFamily.exactRecoveryRate_ge_one_sub_deceptiveRate`.

Theorem 9.3 (Positive exact recovery rate under the threshold)**.** *If the target predictor is realized and*

$$|\mathrm{Code}(H)| \cdot (|X| - 1)^m < |X|^m,$$

then the exact recovery rate is strictly positive.

This is formalized as `EncodedFamily.exactRecoveryRate_pos_of_bound_lt`.

This is still not a full PAC-style theorem under arbitrary distributions. But it does close the uniform finite-sampling gap between "some good sample exists" and "good samples occupy an explicitly bounded fraction of the sample space."

10 Finite Uniform Success Bounds

The next cleanup step is to express the finite uniform rate bound using the single-step miss ratio

$$\rho_X := \frac{|X| - 1}{|X|}.$$

For any nonempty finite input domain, $\rho_X < 1$, so the finite deceptive rate decays exponentially in the sample length m. The Lean layer now also records the elementary order facts behind this slogan:

$$0 \le \rho_X \le 1, \qquad m \le n \Rightarrow \rho_X^n \le \rho_X^m.$$

These are formalized as `uniformMissRatio_nonneg`, `uniformMissRatio_le_one`, and `uniformMissRatio_pow_le_`. The resulting finite-code failure term is antitone in the sample length:

$$m \le n \Rightarrow |\mathrm{Code}(H)|\rho_X^n \le |\mathrm{Code}(H)|\rho_X^m.$$

This is formalized as `EncodedFamily.codeMul_uniformMissRatio_pow_le_of_le`, with the bit-budget specialization `BitEncodedClassifierFamily.bitBudget_uniformMissRatio_pow_le_of_le`.

The module `FiniteUniformSuccess.lean` packages the previous rate theorem into this cleaner interface.

Theorem 10.1 (Miss-ratio deceptive bound)**.** *Let H be an encoded family over a nonempty finite input domain X. Then*

$$\mathrm{rate}_{\mathrm{deceptive}} \le |\mathrm{Code}(H)| \cdot \rho_X^m.$$

This is formalized as `EncodedFamily.deceptiveRate_le_codeMul_uniformMissRatio_pow`.

Theorem 10.2 (Uniform non-deceptive-rate lower bound)**.** *For the same encoded family,*

$$\mathrm{rate}_{\mathrm{nondeceptive}} \ge 1 - |\mathrm{Code}(H)| \cdot \rho_X^m.$$

This is formalized as `EncodedFamily.nondeceptiveRate_ge_one_sub_codeMul_uniformMissRatio_pow`, with epsilon and strict-positivity corollaries. Unlike exact recovery, this statement does not require the target to be realized by the family; it is the pure finite complement of the deceptive-sample upper bound. The monotone form `EncodedFamily.nondeceptiveRate_ge_one_sub_codeMul_uniformMissRatio_pow_c` also says that if $m \le n$, then the weaker m-sample lower bound remains valid for the n-sample non-deceptive rate.

Theorem 10.3 (Uniform exact recovery lower bound)**.** *If the target predictor is realized by the encoded family, then*

$$\mathrm{rate}_{\mathrm{exact\ recovery}} \ge 1 - |\mathrm{Code}(H)| \cdot \rho_X^m.$$

This is formalized as `EncodedFamily.exactRecoveryRate_ge_one_sub_codeMul_uniformMissRatio_pow`. Its monotone companion `EncodedFamily.exactRecoveryRate_ge_one_sub_codeMul_uniformMissRatio_pow_of` gives the same sample-extension principle for exact recovery whenever the target is realized by the encoded family.

Theorem 10.4 (ε-style corollary)**.** *If*

$$|\mathrm{Code}(H)| \cdot \rho_X^m \le \varepsilon,$$

then the exact recovery rate is at least $1 - \varepsilon$.

This is formalized as `EncodedFamily.exactRecoveryRate_ge_one_sub_of_codeMul_uniformMissRatio_pow_le`.

The same module now specializes these bounds to explicitly bit-encoded Boolean families. If F is encoded by s bits, then $|\mathrm{Code}(F)| = 2^s$, so the uniform deceptive rate is bounded by $2^s\rho_X^m$, the non-deceptive rate is at least $1 - 2^s\rho_X^m$, and if the target is realized by F, the exact ERM recovery rate is at least

$$1 - 2^s\rho_X^m.$$

This is formalized as `BitEncodedClassifierFamily.deceptiveRate_le_bitBudget_uniformMissRatio_pow` and `BitEncodedClassifierFamily.nondeceptiveRate_ge_one_sub_bitBudget_uniformMissRatio_pow` and `BitEncodedClassifierFamily.exactRecoveryRate_ge_one_sub_bitBudget_uniformMissRatio_pow`, with epsilon and positivity corollaries plus regression wrappers. The point is not a P versus NP lower bound by itself; it is the exact finite bridge any grassroots compression route must instantiate once it claims an explicit s-bit predictor family.

The module also includes `BitEncodedClassifierFamily.exactRecoveryRate_pos_of_bitBudget_bound_lt`, which keeps the strict-positivity conclusion directly tied to the natural cardinality threshold $2^s(|X| - 1)^m < |X|^m$. Specializing this to the two-point threshold produces `BitFamilyUniformSuccessBinary.lean`: on any finite two-point input domain, and in particular on `Bool`, the simple condition $s < m$ implies a strictly positive uniform exact-recovery rate for every target realized by the s-bit family.

The Boolean specialization is now quantitative as well. Since

$$\rho_{\texttt{Bool}} = \frac{1}{2},$$

the same module proves

$$\mathrm{rate}_{\mathrm{deceptive}} \le 2^s \left(\frac{1}{2}\right)^m$$

and

$$\mathrm{rate}_{\mathrm{nondeceptive}} \ge 1 - 2^s \left(\frac{1}{2}\right)^m$$

and, for every target realized by the s-bit family,

$$\mathrm{rate}_{\mathrm{exact\ recovery}} \ge 1 - 2^s \left(\frac{1}{2}\right)^m.$$

It also records the corresponding epsilon and positivity corollaries. Thus the binary layer now has both the simple threshold $s < m$ and the explicit uniform failure term $2^s(1/2)^m$.

The same result is now also packaged in sample-margin form. If the sample length satisfies

$$s + k \le m,$$

then

$$2^s \left(\frac{1}{2}\right)^m \le \left(\frac{1}{2}\right)^k,$$

and therefore

$$\mathrm{rate}_{\mathrm{nondeceptive}} \ge 1 - \left(\frac{1}{2}\right)^k.$$

For every realized target, the exact ERM recovery rate also satisfies

$$\mathrm{rate}_{\mathrm{exact\ recovery}} \ge 1 - \left(\frac{1}{2}\right)^k.$$

This is formalized as `BitEncodedClassifierFamily.bool_bitBudget_pow_le_margin_pow` and `BitEncodedClassifierFamily.nondeceptiveRate_ge_one_sub_bool_margin_pow` and `BitEncodedClassifierF` The theorem is often the most convenient downstream form: every extra Boolean sample beyond the bit budget halves the remaining uniform failure allowance. There is also an epsilon form: if $(1/2)^k \le \varepsilon$, then under the same margin hypothesis

$$\mathrm{rate}_{\mathrm{nondeceptive}} \ge 1 - \varepsilon \quad \text{and, for realized targets,} \quad \mathrm{rate}_{\text{exact recovery}} \ge 1 - \varepsilon.$$

This is formalized as `BitEncodedClassifierFamily.bool_bitBudget_pow_le_of_margin_pow_le` and `BitEncodedClassifierFamily.nondeceptiveRate_ge_one_sub_of_bool_margin_pow_le` and `BitEncodedClassifierFamily.exactRecoveryRate_ge_one_sub_of_bool_margin_pow_le`. The strict positive-margin form is now proved too. If $0 < k$ and $s + k \le m$, then

$$2^s \left(\frac{1}{2}\right)^m < 1.$$

Consequently the non-deceptive uniform rate is strictly positive, and for any realized target the exact ERM recovery rate is strictly positive. In Lean these are `BitEncodedClassifierFamily.bool_bitBudget_pow_lt_` `BitEncodedClassifierFamily.nondeceptiveRate_pos_bool_of_positive_margin`, and `BitEncodedClassifie` The concrete $s < m$ case is also recorded as `BitEncodedClassifierFamily.bool_bitBudget_pow_lt_one_of_lt_s` and `BitEncodedClassifierFamily.nondeceptiveRate_pos_bool_of_lt_sampleSize`.

The clocked *Kpoly* counting bridge has the corresponding strict positive-margin success statements. A clocked s-bit Boolean family with $0 < k$ and $s + k \le m$ has strictly positive non-deceptive rate for every target; if the target is realized by the family, it also has strictly positive exact ERM recovery rate. These are formalized as `ClockedBitCodeFamily.nondeceptiveRate_pos_bool_of_positive_margin` and `ClockedBitCodeFamily.exactRecoveryRate_pos_bool_of_positive_margin`. The concrete $s < m$ existence theorems are `ClockedBitCodeFamily.exists_bool_nondeceptiveSample_of_lt_sampleSize` and `ClockedBitCodeFamily.exists_bool_sample_empiricalRiskPredictor_eq_target_of_lt_sampleSize`. The recovery theorem still consumes a realized clocked bit-family; it does not assert that the manuscript's switched decoder has such a small realization.

This is still uniform finite-sample learning rather than full distributional generalization. But it is the first version of the grassroots story with an explicit success lower bound of the form "one minus an exponentially decaying failure term."

11 Finite Weighted Agreement and Family Bounds

The next honest move is to relax the uniform-sampling assumption while still staying fully finite. Instead of sampling uniformly from a finite input domain, we let one finite probability mass function μ weight the inputs.

The module `FiniteIIDAgreement.lean` handles the one-predictor case. For one target f and one predictor h, it defines:

1. the one-step agreement mass of h with f under μ,
2. the product mass of a length-m sample under independent draws from μ,
3. the total mass of samples on which h agrees with f everywhere.

Theorem 11.1 (Exact weighted agreement law)**.** *For any finite input type and finite PMF μ,*

$$\mathrm{mass}_\mu(\textit{all-}m\textit{-point samples consistent with } h) = \left(\mathrm{agreementMass}_\mu(f, h)\right)^m.$$

This is formalized as `consistentSampleMass_eq_agreementMass_pow`.

Theorem 11.2 (Weighted decay corollary)**.** *If the one-step agreement mass is at most q, then the total mass of length-m consistent samples is at most q^m.*

This is formalized as `consistentSampleMass_le_pow_of_agreementMass_le`.

The new module `FiniteIIDFamilyBound.lean` lifts this to the encoded-family level.

Theorem 11.3 (Weighted deceptive-family union bound)**.** *Let H be a finite encoded family and μ a finite PMF on the input type. Then the total μ^m-mass of deceptive samples is at most*

$$\sum_{c \in \mathrm{BadCodes}(H,f)} \left(\mathrm{agreementMass}_{\mu}(f, \mathrm{decode}(c))\right)^m.$$

This is formalized as `EncodedFamily.deceptiveSampleMass_le_badCodeAgreementMassSum`.

Theorem 11.4 (Uniform weighted family corollary)**.** *If every bad code in the family has one-step agreement mass at most q, then the total deceptive mass is at most*

$$|\mathrm{Code}(H)| \cdot q^m.$$

This is formalized as `EncodedFamily.deceptiveSampleMass_le_codeCard_mul_pow_of_agreementMass_le`.

This is the first genuinely family-level finite distributional theorem in the grassroots development. The next real probabilistic step is no longer a union bound itself, but a recovery or generalization theorem built on top of this weighted family bound.

12 Finite Weighted Recovery Bounds

The module `FiniteIIDRecovery.lean` closes the next natural loop. It turns the weighted deceptive-mass bound into a weighted success statement for the chosen ERM predictor.

Theorem 12.1 (Weighted exact recovery dominates the nondeceptive mass)**.** *If the target predictor is realized by the encoded family, then the total μ^m-mass of exact ERM recovery is at least*

$$1 - \mathrm{deceptiveMass}_{\mu}.$$

This is formalized as `EncodedFamily.exactRecoverySampleMass_ge_one_sub_deceptiveSampleMass`.

Theorem 12.2 (Weighted exact recovery from bad-code agreement masses)**.** *If the target predictor is realized by the encoded family, then the exact recovery mass is at least*

$$1 - \sum_{c \in \mathrm{BadCodes}(H,f)} \left(\mathrm{agreementMass}_{\mu}(f, \mathrm{decode}(c))\right)^m.$$

This is formalized as `EncodedFamily.exactRecoverySampleMass_ge_one_sub_badCodeAgreementMassSum`.

Theorem 12.3 (Uniform weighted exact recovery corollary)**.** *If every bad code has one-step agreement mass at most q, then the exact recovery mass is at least*

$$1 - |\mathrm{Code}(H)| \cdot q^m.$$

This is formalized as `EncodedFamily.exactRecoverySampleMass_ge_one_sub_codeCard_mul_pow_of_agreement`

At this point the grassroots development has reached a fully finite, distribution-weighted recovery theorem for ERM over encoded families. The next step is to lift this from exact recovery on a finite sample space to a genuine finite-class generalization statement.

13 What Still Counts as the Same Route

At this stage the grassroots theory can also say something more conceptual, but in a machine-checked way. If a proposed rescue still wants to be *the same route* as the current switching/ERM program, then it cannot merely gesture at weakness, GSLTs, or observer-relative semantics. It must still instantiate the existing finite learning interface.

The new module `SameRouteInterface.lean` packages exactly that point. A *same-route family* is just:

1. one available input surface AvailableInput(P, B, G),
2. one s-bit encoded boolean predictor family on that surface,
3. one target predictor realized by some code in that family.

Theorem 13.1 (Same-route weighted recovery certificate)**.** *Let F be an s-bit encoded classifier family and suppose the target predictor is realized by some code in F. If every bad code has one-step agreement mass at most q, then the exact ERM recovery mass is at least*

$$1 - 2^s q^m.$$

This is formalized as `BitEncodedClassifierFamily.exactRecoverySampleMass_ge_one_sub_bitBudget_mul_po`

The companion module `BitFamilyWeightedRecovery.lean` packages the same fact in the form needed by downstream route interfaces: if the explicit failure term $2^s q^m$ is at most a chosen tolerance ε, then exact ERM recovery has sample mass at least $1 - \varepsilon$. In Lean this is `BitEncodedClassifierFamily.exactRecoverySampleMass_ge_one_sub_of_bitBudget_mul_pow_le`, with a regression wrapper in `BitFamilyWeightedRecoveryRegression.lean`.

The point is not that this theorem proves the current switching step. The point is that it now marks the boundary of the current route. A weakness/GSLT idea that still intends to be a *repair* of the present program must hand over this sort of certificate: an explicit visible input surface, an explicit small code space, and explicit agreement-mass control for the bad codes. If it does not, then it is no longer filling a gap in the current route. It is starting a different route.

The agreement-mass hypothesis is not a bookkeeping side condition. The new module `BadCodeAgreementObstruc` proves the support-level test: if a bad code agrees with the target on every positive-mass input, its agreement mass is exactly 1. Therefore no strict $q < 1$ recovery package can exist unless each bad code disagrees with the target somewhere on positive sampling mass. In Lean this is recorded generically as

`BitEncodedClassifierFamily.not_badCodeAgreementBound_of_agrees_on_support`

and, contrapositively, as

`BitEncodedClassifierFamily.exists_pos_mass_disagreement_of_badCode_agreementBound_lt_one`.

The atomic special case is also pinned: for $\mu = \delta_x$, any bad code that agrees with the target at x has agreement mass exactly 1. This is recorded as

`BitEncodedClassifierFamily.not_badCodeAgreementBound_pure_of_agrees`

and at the exact ZAB interfaces as

`not_ExactZABERMRecoveryData_of_pure_badCode_agrees`, `not_CanonicalZABERMRecoveryData_of_pure_ba`

Equivalently, under a pure-atom bound with $q < 1$, any code that matches the target at that atom must be the target function globally. Thus finite-image data for the ERM wrapper is still insufficient: the manuscript must also prove a real support/disagreement theorem for the actual switched bad codes and the measure being used.

14 Exact Post-Switch Summary Compression

The next optimistic cleanup is to isolate the exact object that still needs to be compressed. The module `ExactSwitchedFamily.lean` now makes that object explicit.

The manuscript-visible post-switch datum is the exact local surface

$$u = (z, a, b).$$

The Lean file separates:

1. the full exact visible surface $u = (z, a, b)$,
2. the invariant projection (z, a),
3. the fork projection $((z, a), b)$.

The main positive result is not yet a proof that the actual switched family is small. It is the next honest theorem underneath that goal.

Theorem 14.1 (Finite summary implies explicit bit budget)**.** *Let G be an indexed Boolean predictor family on some input type U. If G factors through a map*

$$\text{view} : U \to V$$

to a finite summary surface V, then G admits a concrete truth-table bit budget of size $|V|$.

This is formalized as `IndexedPredictorFamily.hasBitBudget_card_of_factorsThrough_fintype`.

So the optimistic burden is now cleaner. It is no longer

> compress the switched family somehow.

It is:

> prove that the switched family depends only on a genuinely small finite summary surface.

The exact post-switch specializations are now also explicit.

Theorem 14.2 (Invariant-view compression kernel)**.** *If the exact switched family on $u = (z, a, b)$ factors through the invariant projection (z, a), then it has an exact visible compression target with bit budget*

$$|Z| \cdot 2^k.$$

This is formalized by `card_invariantVisiblePostSwitchSurface` together with `exactVisibleCompressionTarge`

Theorem 14.3 (Fork-view compression kernel)**.** *If the exact switched family factors through the fork projection $((z, a), b)$, then it has an exact visible compression target with bit budget*

$$|Z| \cdot 2^k \cdot 2^k.$$

This is formalized by `card_forkVisiblePostSwitchSurface` together with `exactVisibleCompressionTarget_of_`

This does *not* defeat the crux. Those summary surfaces may still be far too large. But it does give the ground-up reduction we wanted: a successful repair must now prove an actually small finite summary theorem, not merely a slogan about switched locality.

The new module `GlobalWeaknessObstruction.lean` makes the first exact Meredith-side pressure point equally explicit. In the current Lean bridge, `distinctionWeakness ev` and `nonDistinctionWeakness ev` are global scalars attached to the whole finite evidence assignment ev. They do not vary with the underlying state $u \in U$.

Theorem 14.4 (Weakness-only factoring is constant)**.** *If a boolean predictor $h : U \to \{\text{false}, \text{true}\}$ factors through either of the current global summaries*

$$\text{distinctionWeakness}(ev) \qquad \textit{or} \qquad \text{nonDistinctionWeakness}(ev),$$

then h is constant on U:

$$\forall u, v \in U,\ h(u) = h(v).$$

This is formalized as `factorsThroughDistinctionWeakness_constant` and `factorsThroughNonDistinctionWeak`
So the current weakness bridge is real semantic infrastructure, but by itself it does not yet instantiate the same-route learning interface. Any GSLT/weakness rescue that still wants to count as the current switching/ERM route must add genuinely per-instance observables or side information beyond these present global summaries.

15 Hyperseed as a Local-View Surface

The existing Hyperseed formalization already provides a clean observer-relative semantics.

Given a perspective P, a budget B, and a guard region G, it defines an *available region*

$$\text{availableRegion}(P, B, G) = \text{observableUniverse}(P) \cap \text{nearEurycosm}(P, B) \cap G.$$

For the grassroots PNP program, this matters because it gives a principled way to talk about what a decoder can even access. Instead of treating locality as an informal phrase, we can work with a typed input surface:

$$\text{AvailableInput}(P, B, G) := \{x : \text{World} \mid x \in \text{availableRegion}(P, B, G)\}.$$

The new Lean module proves the corresponding encoded-family bound on such an available input surface:

Theorem 15.1 (Available-region code-budget bound)**.** *An s-bit classifier family over one Hyperseed available region still realizes at most 2^s classifiers.*

This is formalized as `card_realized_availableFamily_le_of_bitBudget`.

The theorem is intentionally modest. It does not claim that the current local view *is* a Hyperseed perspective. It says that Hyperseed already gives us a good general language for bounded observational access, and that this language can interoperate with the encoded-hypothesis-class infrastructure.

16 Operational Distinction and Weakness

My own mathematical and computationalist intuition is that if there is an elegant route through the real blockers, it will likely come from a semantics of *distinguishability* rather than from ad hoc local-rule counting alone. That is the reason the Meredith lane is worth treating as part of the grassroots background theory, at least optimistically.

At the abstract level, the bridge has three layers:

1. a GSLT supplies an operational state space together with its bisimulation quotient;
2. context-decorated HML supplies a language of experiments or observations;
3. quantale weakness measures the mass of distinction events on that quotient.

In slogan form:

> operational behavior gives the ontology, HML gives the observables, and weakness turns distinguishability into evidence.

This is now more than a philosophical analogy in the local Lean state.

Theorem 16.1 (Depth-bounded observational equivalence)**.** *For each modal depth n, the Lean development defines a relation*

$$\mathrm{hmlEquivUpTo}_n(t, u),$$

meaning that no context-decorated HML formula of modal depth at most n distinguishes t from u.

This is formalized in `GSLT/Logic/LogicalMetric.lean`. The same file also defines a binary-valued depth-n logical-distance approximation

$$d_n(t, u) \in \{0, 1\},$$

and proves the corresponding pseudoultrametric inequality.

Theorem 16.2 (Finite-depth witness forces quotient-level distinction under adequacy)**.** *Assume the sound direction of Meredith's adequacy theorem for a GSLT S. If some depth-n context-decorated HML formula distinguishes t from u, then t and u are genuinely distinct in the bisimulation quotient of S. Equivalently, if*

$$d_n(t, u) = 1,$$

then the bisimulation classes $[t]$ and $[u]$ are different.

This is now formalized in `GSLT/Meredith/WeaknessBridge.lean` as `distinguishingWitness_implies_distingui` and `logicalDistanceApprox_eq_one_implies_classes_ne_of_adequacySound`.

Theorem 16.3 (Weakness on the bisimulation quotient)**.** *For a finite bisimulation quotient U equipped with a quantale-valued weight assignment μ, the Lean bridge defines the weakness of the distinction event*

$$\mathrm{weakness}_\mu(\mathrm{distinctionEvent}(U)),$$

interpreting operational distinguishability as an evidence quantity.

This is the content of `GSLT/Meredith/WeaknessBridge.lean`, building on `GSLT/Meredith/Bisimulation.lean`. In the finite probabilistic case, this has the expected logical-entropy shape.

The new concrete step is that the abstract minimal-context interface now has an actual process-calculus instance.

Theorem 16.4 (Concrete rho-calculus minimal-context bridge)**.** *The rho-calculus evaluation contexts from* `Languages/ProcessCalculi/RhoCalculus/Context.lean` *now induce a concrete* `HasMinimalContexts` *instance for the rho GSLT, and for each rho context* K*,*

$$\mathrm{contextStep}(p, K, q) \iff p \rightsquigarrow [K]\, q.$$

This is formalized in `GSLT/Meredith/RhoMinimalContext.lean`. That file also proves concrete HML diamonds for matching COMM interactions.

For the PNP grassroots route, the optimistic use of this material is not that it proves the switching theorem by itself. Rather, it offers a clean semantics for coarse observational views. If a decoder is only sensitive to finite-depth local experiments, then one can hope to represent that by an HML fragment and then ask how much evidence survives under the induced quotient. That is exactly the sort of elegant abstraction that may help disentangle the real compression/locality story from the current proof's more brittle prose. The bridge is now one step tighter than before: finite-depth observational separation no longer just lives as a metric-like approximation, but already certifies distinct quotient classes once adequacy-soundness is available.

The honest limit is equally important: this is still background theory. It does *not* yet produce the switched small class, the generalization theorem, or the K_{poly} contradiction step. But it is now a real machine-checked background layer, not merely an idea. The new obstruction above sharpens this limit: the currently formalized weakness summaries are quotient-level evidence scalars, not a per-instance predictor surface. So any semantic rescue that remains close to the current switching/ERM route still needs an extra layer that turns operational semantics into state-indexed encoded observables.

The next obstruction is sharper still. The new module `InfinitaryHMLObstruction.lean` proves:

1. if $\mathrm{MinimalContext}(S)$ is infinite, then $\mathrm{HMLFormula}(S)$ is infinite;
2. under the same hypothesis, even the depth-1 fragment $\mathrm{DepthFragment}_1(S)$ is infinite;
3. in the concrete rho instance, there is an explicit infinite ladder of certified minimal contexts, so the rho depth-1 HML fragment is already infinite;
4. therefore no finite code space can surject onto the current exact depth-1 probe family, abstractly or in the concrete rho instance.

So the current Meredith semantic layer is now pinned down even more tightly: before one can talk about polylog(m)-bit encoded predictor families, one first needs a separate theorem that compresses or finitizes the relevant observational probe family. The present HML/context layer does not supply that for free.

The Meredith-side trilemma is now simple.

1. Use the current weakness summaries: then the predictor is constant.
2. Use the current exact HML/context probes: then the observable family is already too large, even at depth 1.
3. Use the current exact present-moment surface directly: then one already encounters powerset-like finite probe slices.

So any close-form repair now has to do something genuinely new. It is no longer enough to say "use current weakness" or "use current HML probes a bit more cleverly." One needs a new compressed observational layer together with an explicit theorem showing that the switched predictor factors through it.

The new module `PresentMomentShattering.lean` tightens this further on a very concrete slice of the current rho semantics. Let state i be a single output on channel i, and let environment S be a flat bag of input guards on the channels in a finite list S. Then the file proves:

1. channel i lies in surfaceChannels$(\text{state}_i, \text{env}_S)$ iff $i \in S$;
2. the concrete external-present-moment pair $(\text{par}(\text{env}_S, \square), i)$ is present iff $i \in S$.

So even before passing to the exact HML layer, the currently formalized state-indexed observational surface already realizes arbitrary finite membership patterns on a natural probe family. This is not yet a full "depth-1 HML shattering" theorem, but it means the large-observable-surface pressure is already visible one layer earlier in the exact present-moment semantics. In fact, the file now packages this as an injective finite-signature map $S \mapsto (i \mapsto \text{observable}_S(i))$ on $\text{Finset}(\text{Fin}\, n)$: distinct finite probe sets yield distinct exact present-moment signatures on the n probe states. Any close-form Meredith rescue now has to compress not only an infinitary exact HML layer, but already a powerset-like concrete probe family in the rho present-moment surface.

17 Kolmogorov Foundations Already Present

The local repository also already contains a substantial Kolmogorov-complexity seed under `Mettapedia/Computabili`

In particular, `Basic.lean` defines:

1. binary strings as `List Bool`,
2. partial algorithms,
3. plain Kolmogorov complexity $C[U](x)$,
4. a universal algorithm built from mathlib's partial-recursive code,
5. the usual invariance theorem for universal algorithms.

This is not yet the time-bounded conditional complexity K_{poly} that the present route uses. But it matters for the grassroots route because it means the eventual K_{poly} development should be an extension of an existing computability layer rather than a fresh bespoke definition dropped into a late-stage contradiction proof.

In practical terms, the next computability milestone is:

> define conditional and polynomial-time-bounded complexity on top of the existing universal-machine layer, instead of axiomatizing them only at the paper level.

18 What the Grassroots Program Needs Next

The current background theory suggests a fairly crisp optimistic roadmap.

18.1 Finite-Class Learning Layer

The ERM existence layer, finite uniform consistency bound, threshold recovery corollary, rational uniform-rate layer, miss-ratio success bound, and weighted one-predictor and finite-family weighted bounds are now in place. The next finite-class learning steps are:

1. a generalization bound stated in terms of $\log |H|$,
2. a probability interface for i.i.d. or distributional assumptions at the family level,
3. concentration-style control beyond the exact finite-space formulas,
4. a clean interface separating statistical assumptions from coding facts.

In other words, the current Lean state already handles:

1. empirical error on finite samples,
2. existence of ERM predictors in nonempty finite encoded families,
3. exact-fit transfer when some code in the family is sample-consistent,
4. a finite uniform bound on the deceptive sample set,
5. exact ERM recovery of the target off that deceptive set,
6. a strict-cardinality threshold guaranteeing the existence of a good sample,
7. a rational upper bound on the deceptive sample rate,
8. a rational lower bound on the non-deceptive sample rate,
9. a rational lower bound on the exact recovery rate,
10. strict positivity of that exact recovery rate under the threshold,
11. explicit miss-ratio lower bounds of the form $1 - |H|\rho_X^m$,
12. an exact weighted one-predictor law of the form $\text{mass} = \alpha^m$,
13. a weighted family union bound of the form $\text{deceptiveMass} \le \sum_{h \in H_{\text{bad}}} \alpha_h^m$,
14. a weighted exact recovery bound of the form $\text{recoveryMass} \ge 1 - \sum_{h \in H_{\text{bad}}} \alpha_h^m$.

What remains is to lift that finite uniform rate layer into a real finite-class generalization theorem.

18.2 Concrete Decoder Subclasses

The encoded-family theorem is only an interface. It still needs a nontrivial source of small classes. The likely optimistic candidates are concrete local decoders on tree-like neighborhoods:

1. belief-propagation style update rules,
2. GF(2)-linear constraint summaries from the VV layer,
3. bounded-depth compositions of those primitives.

The first concrete instance of that second lane now exists. The new module `AffineColumnFamily.lean` formalizes affine GF(2)-style predictors on the retained VV column bits and proves:

> if the exact switched family collapses to affine parity tests on the column view a, then the exact visible compression target is only $k + 1$ bits.

The key formal statement is `exactVisibleCompressionTarget_of_realizedByExactAffineColumnFamily`. This does not prove that the real switched family is affine. It does give the optimistic route its first genuinely nontrivial concrete landing zone below the full truth-table class.

The next concrete step is now also formalized. The new module `AffineFeatureFamily.lean` allows a bounded number r of affine GF(2)-style VV-column features, followed by an arbitrary truth table on the resulting r feature bits. Its exact raw bit budget is

$$r(k + 1) + 2^r.$$

The key formal statement is `exactVisibleCompressionTarget_of_realizedByExactAffineFeatureFamily`. So if the switched family collapses not to a single affine probe but to a bounded affine-feature algebra, the code budget is still explicit and still far below the full truth-table class on $u = (z, a, b)$.

The next concrete step is sharper and more useful. The new module `SparseThresholdAffineFamily.lean` keeps the same r affine GF(2)-style VV-column features but replaces the full truth table by a much smaller combiner: an r-bit mask selecting active features and an r-bit threshold code. Its exact raw bit budget is now linear:

$$r(k + 3).$$

The key formal statement is `exactVisibleCompressionTarget_of_realizedByExactSparseThresholdAffineFam`
So if the switched family collapses to a sparse-threshold affine algebra on the retained VV column view, the optimistic route now has a concrete candidate class whose coding cost no longer contains the exponential 2^r blow-up.

If one of these affine / sparse-threshold families can be shown to encode the actual switched predictors uniformly in polylog(m) bits, then the switching theorem has a plausible landing zone.

The next refinement is now also formalized. The new module `AffineDecisionListFamily.lean` keeps the same r affine GF(2)-style VV-column features, but replaces the combiner by a fixed-order decision list: each feature carries one response bit, and there is one default output bit if no feature fires. Its exact raw bit budget is

$$r(k + 2) + 1.$$

The key formal statement is `exactVisibleCompressionTarget_of_realizedByExactAffineDecisionListFamily`
So if the switched family collapses to an ordered affine decision-list algebra on the retained VV column view, the optimistic route now has an even smaller explicit candidate class than the sparse-threshold family.

The next layer is now also formalized. The new module `ExactAffineRecovery.lean` turns those four exact-surface candidate classes into direct weighted finite-sampling recovery theorems. Once the target lies in one of the concrete exact-surface affine families, the existing recovery backbone from `SameRouteInterface.lean` applies immediately with one of the four explicit budgets

$$k + 1, \qquad r(k + 1) + 2^r, \qquad r(k + 3), \qquad r(k + 2) + 1.$$

So the current small-class ladder is no longer only a compression interface. It is now a direct recovery interface too.

The next big refinement is now also formalized. The new module `SharedAffineFeatureFamilies.lean` isolates the regime where one fixed affine feature basis is shared across the switched family and only

the downstream combiner varies. In that case the feature extractor no longer has to be re-encoded per predictor, so the exact visible compression target drops to the combiner alone:

$$2^r, \qquad 2r, \qquad r+1$$

for the shared truth-table, shared sparse-threshold, and shared decision-list regimes respectively. The same module also proves the corresponding weighted exact-recovery lower bounds on the exact post-switch surface.

This is the first genuinely sharp improvement in the optimistic lane. If the real switched family uses a common low-complexity feature extractor and varies only in a light combiner, then the coding burden is much smaller than in the previous per-predictor affine-family models.

The next concrete step is now also formalized. The new module `ExactABDecisionListFamily.lean` fixes the shared extractor to the raw exact visible bits themselves: concatenate the retained VV column bits a with the side-channel bits b, then run a fixed-order decision list on those $2k$ raw bits. This yields a completely concrete exact-surface candidate class with budget

$$2k+1.$$

The key point is that the feature extractor is no longer existential or parameterized. It is the literal raw exact visible surface restricted to the bit coordinates (a, b). The same module also proves the corresponding weighted exact-recovery lower bound for that family.

The route is now factored one step more cleanly. The new modules `ABVisibleSurface.lean` and `ABDecisionListRoute.lean` isolate the reduced raw visible surface (a, b) as an explicit quotient candidate and prove the pullback statement one actually wants: if the switched family factors through that reduced surface and the reduced family is realized by fixed-order decision lists there, then the exact-surface compression and weighted recovery theorems follow immediately. This is also the natural insertion point for any future observational-equivalence argument: a semantic quotient can help justify the map to (a, b), but it does not need to alter the learning-theoretic backbone once that map is fixed. The same route file now also proves the slightly stronger lift: if an exact-surface family is invariant under the (a, b) quotient, then a canonical reduced family can be reconstructed and the same $2k+1$-bit decision-list target follows from that invariance alone.

The matching strict-subclass fact is now machine-checked as well. `ABDecisionListObstruction.lean` defines the fixed-order decision-list bit family directly on the reduced raw surface and proves that realization by that family gives a $2k+1$-bit budget on `ABVisibleSurface` k. For $k > 0$, Lean proves

$$2k+1 < 2^{2k} = |\texttt{ABVisibleSurface}\ k|.$$

Hence the raw decision-list route cannot realize the full Boolean rule family on (a, b). The same obstruction lifts through the exact-surface pullback: if the exact switched family factors through (a, b) but the reduced family is still fully expressive, then the raw exact-surface decision-list class cannot realize it. This is grassroots progress rather than a final blocker: it says exactly which finite candidate class has been bounded, and leaves broader randomized, approximate, and *Kpoly* bridges outside the claim.

The raw (a, b) route is now richer than a single fixed-order decision list. The new module `ExactABAffineFamilies.lean` reuses the existing affine machinery on the same raw extractor and proves exact-surface compression and weighted recovery bounds for three further candidate classes on the raw (a, b) bits:

$$r((2k)+1)+2^r, \qquad r((2k)+3), \qquad r((2k)+2)+1$$

for bounded affine features, sparse-threshold affine predictors, and affine decision lists respectively. So the reduced raw surface is now a whole candidate-family ladder, not just one isolated family.

The next refinement is now in place as well. The new module `SharedExactABFeatureFamilies.lean` adds the shared-basis collapse on that same raw (a, b) surface. If one fixed affine basis on the $2k$ raw bits is reused across the whole switched family and only the downstream combiner varies, the exact visible compression target drops all the way to combiner-only budgets:

$$2^r, \qquad 2r, \qquad r + 1$$

for arbitrary truth tables, sparse-threshold combiners, and fixed-order decision lists respectively. So the raw (a, b) quotient now supports both a per-predictor affine ladder and a sharper shared-basis ladder.

That bridge is now factored explicitly too. The new module `SharedABFeatureRoutes.lean` proves that if the switched family factors through the reduced raw surface (a, b) and the reduced family is realized by one of those shared-basis raw affine classes, then the exact-surface family inherits the same combiner-only budget immediately. This is the sharpest current handoff point between any future observational quotient theorem and the small-class recovery machinery. The same file now also proves the invariance version: if the exact-surface family is already invariant under the raw (a, b) quotient, then the canonical reduced family inherits those same shared-basis combiner-only budgets without any separately supplied factorization witness.

The route-coverage map now records the matching strict-subclass obstruction as well. The module `SharedABFeatureObstruction.lean` proves that if the reduced raw family on (a, b) is still surjective onto all Boolean rules, then no fixed shared raw affine basis can realize it below the reduced truth-table threshold. Lean proves this for the three combiner budgets

$$2^r, \qquad 2r, \qquad r + 1$$

corresponding to arbitrary affine-feature truth tables, sparse-threshold combiners, and fixed-order decision lists. The same result is lifted back to the exact surface: if the exact switched family factors through (a, b) and the reduced family is fully expressive, then the corresponding shared exact class cannot realize it below those budgets. This adds honest narrow entries to the `PNPKpolySubrepairClass` ledger alongside the finite-image budget, representative-index, quotient-code, transport, and reduced-AB finite-image/quotient pullback entries, while leaving the broad `kpolyCompressionBridge` open; it rules out full reduced-rule expressivity for named shared classes, not every possible manuscript-faithful small-class theorem.

The target statement itself is now packaged in that style too. The new module `SharedABTargetInterface.lean` bundles the current route bottleneck into explicit data packages: one shared raw (a, b) feature basis, one quotient invariance hypothesis, and one realization hypothesis for the lifted reduced family. From that package it derives the exact visible compression target with budget 2^r, $2r$, or $r + 1$ depending on the combiner class. This is the cleanest current formal statement of what the optimistic route actually needs.

There is now also a clean backward step from that generic target statement back toward the most concrete current route. The module `CanonicalABTargetRoute.lean` specializes the shared raw (a, b) feature basis to the canonical coordinate probes on the raw visible bits. In that specialization, the generic shared-basis decision-list target collapses exactly to the older raw (a, b) decision-list route. So the current formal burden can now be read in two equivalent ways:

1. as a generic shared-basis target interface on the reduced raw surface,
2. or as quotient invariance plus realization by fixed-order decision lists on the literal raw visible bits.

This is the clearest backward-chaining reach from the cleaned-up target statement back to the existing concrete work.

That concrete work is now packaged directly as well. The new module `CanonicalABCandidateInterface.lean` takes the canonical raw-bit route and bundles it into one explicit candidate-data object: quotient invariance under the raw (a, b) projection together with realization of the lifted reduced family by fixed-order decision lists on the raw visible bits. From that single package it derives both the exact visible compression target with budget $2k + 1$ and, for each indexed predictor in the family, the matching weighted exact-recovery lower bound through the already-existing raw decision list bit family. So the optimistic burden is now even sharper: produce one such candidate-data witness, then prove the needed agreement-mass estimate.

That burden has now been reduced one step further. The new module `CanonicalABCodeWitness.lean` proves that if the switched family is exactly the family induced by an explicit assignment of one raw (a, b) decision-list code to each index, then the canonical candidate-data package, the $2k + 1$ compression target, and the per-index recovery theorem all follow automatically. So the route bottleneck is now close to its simplest concrete form: show that the real switched family equals one such code-induced family, then prove the corresponding agreement-mass estimate.

The last generic layer is now also in place. The new module `CanonicalABRecoveryInterface.lean` packages exactly the remaining generic information needed by the route:

1. one explicit raw (a, b) decision-list code assignment per index,
2. equality between the switched family and the induced code family,
3. one agreement-mass upper bound.

From that single recovery-data package, the $2k + 1$ compression target and the weighted exact-recovery lower bound both follow automatically. At that point, the remaining work is no longer background theory; it is the actual candidate-specific switched-family theorem.

The next manuscript-facing extension is now in place as well. The paper's real local input is not just (a, b), but $u = (z, a, b)$, where z comes from one fixed shared extractor on the masked block. The new module `ExactZABDecisionListFamily.lean` formalizes the corresponding candidate class directly: for any shared extractor $\mathsf{zfeat} : Z \to \{0, 1\}^r$, one concatenates $\mathsf{zfeat}(z)$, a, and b, then applies the same fixed-order decision-list family as before. The exact-surface budget is then $r+2k+1$, and the matching weighted exact-recovery bound is proved on that larger visible surface.

That new route is then packaged in `ExactZABTargetInterface.lean`. The burden is exactly what one would want it to be: prove that the switched family is realized by fixed-order decision lists on $(\mathsf{zfeat}(z), a, b)$, then prove one bad-code agreement-mass estimate. No extra generic machinery is needed beyond that package. The same package now also exposes the finite-image content of the route explicitly: from the target data it derives a finite predictor cover, finite representative-index cover, and finite quotient-code presentation of size at most 2^{r+2k+1}. Thus the concrete exact-ZAB route does not hide the current *Kpoly* repair target inside the compression-target abbreviation.

Finally, the manuscript's constructive wrapper now plugs into the route directly. The new module `BitFamilyERM.lean` turns any fixed bit-encoded local class into an indexed ERM-selected family together with its selected code at each sample. Then `ExactZABERMRoute.lean` specializes that statement to the shared $(\mathsf{zfeat}(z), a, b)$ decision-list class: if the wrapper is honestly specified as ERM over that class, then the selected code itself is the route witness, and the compression target follows immediately. The same ERM route now exposes the finite predictor cover, representative-index cover, and quotient-code presentation for the ERM-selected family itself. At that point, the remaining burden is even sharper: identify the right shared extractor zfeat, show the wrapper is using that class, and control the agreement mass of the bad codes.

There is now one more collapse on top of that manuscript-facing route. The new module `SharedExactZABFeatureFamilies.lean` assumes that the local bits $(\mathsf{zfeat}(z), a, b)$ first pass through one fixed shared affine-feature basis of width p, and only the final combiner varies across predictors. On that shared-basis route, the exact-surface budgets fall to 2^p for arbitrary truth tables, $2p$ for sparse thresholds, and $p + 1$ for fixed-order decision lists. Then `SharedExactZABTargetInterface.lean` packages the corresponding target and recovery interfaces, and `SharedExactZABERMRoute.lean` shows that if the wrapper is honestly ERM over that shared-basis decision-list class, then the selected code itself again is the route witness, now with the sharper $p+1$ budget. The shared-basis exact route now also exposes the same finite-image data as the direct exact route. `SharedExactZABTargetInterface.lean` derives finite predictor covers, representative-index covers, and quotient-code presentations for the affine-feature, sparse-threshold, and shared decision-list variants. Then `SharedExactZABFeatureERMInterface.lean` packages the final ERM-facing affine-feature and sparse-threshold forms, and `SharedExactZABERMInterface.lean` packages the final ERM-facing shared decision-list form. So at the shared-basis endpoint, one sample assignment, one ERM-equality witness, and one bad-code agreement bound already yield both the weighted recovery inequality and the explicit finite-image payload.

So the remaining burden is now split cleanly into two candidate routes on the real local surface $u = (z, a, b)$:

1. the direct decision-list route on $(\mathsf{zfeat}(z), a, b)$, with budget $r + 2k + 1$,
2. the shared-basis route, where one fixed affine basis on $(\mathsf{zfeat}(z), a, b)$ reduces the decision-list budget to $p + 1$.

The second route is now the sharper optimistic landing point.

The direct route has now been backward-chained as far as it can go generically. The new module `CanonicalZABTargetRoute.lean` specializes the shared-basis statement to the canonical coordinate basis on the raw exact bits, so the shared-basis route collapses back to the direct exact decision-list route. Then `CanonicalZABCandidateInterface.lean` packages that direct route as one concrete candidate object, `CanonicalZABCodeWitness.lean` reduces it to one explicit exact decision-list code assignment per index, and `CanonicalZABRecoveryInterface.lean` packages the last direct-route ingredient: one agreement-mass upper bound. So even on the less sharp $r + 2k + 1$ route, the remaining burden is now completely explicit. Moreover, `CanonicalZABERMRoute.lean` removes the last administrative gap between the honest ERM wrapper and that final direct route: once the wrapper is really ERM over the raw exact $(\mathsf{zfeat}(z), a, b)$ decision-list class, the canonical code witness is exactly the ERM-selected code. The new module `CanonicalZABERMInterface.lean` packages the same conclusion in the most manuscript-facing form: one sample assignment, one proof that the switched family is exactly that ERM wrapper, and one bad-code agreement bound. The canonical candidate, explicit-code, recovery, and final ERM layers now carry the same finite predictor cover, representative-index cover, and quotient-code objects directly. This discharges the finite-image bookkeeping for the actual canonical exact decision-list family conditional on an explicit code assignment or equality to the ERM wrapper; it does not prove that the manuscript's broader switched predictor family has such a realization.

There is now also one fully concrete specialization of that exact route. `BitVecZABVisibleSurface.lean` fixes the case where the local datum is already a bitvector $z : \mathrm{BitVec}\, r$, so the full visible input (z, a, b) becomes one raw bitvector of length $r + 2k$. Then `BitVecZABERMInterface.lean` specializes the final exact ERM interface to the identity extractor on that raw full visible bit surface. So in that case the remaining burden is no longer "choose zfeat"; it is only "choose the samples, prove equality to the ERM wrapper, and control the bad codes." The new module `ProjectedZABERMInterface.lean`

narrows the same burden in a different concrete way: when z is bit-valued, it is enough to choose finitely many coordinates of z as the local summary extractor. Both concrete ERM specializations now also expose the three finite-image objects directly, so the remaining manuscript obligation is not hidden behind the identity or projection wrappers.

18.3 Bridging to the Full PNP Route

Only after those layers are built does it make sense to reconnect to the full proof program:

1. define the distributional ensemble cleanly,
2. define local views as typed accessible surfaces,
3. prove the switched family lies in a small encoded subclass,
4. connect success bounds to a properly formalized K_{poly},
5. finally compare with the upper bound under $P = NP$.

19 Current Status

At present, the grassroots side has several real machine-checked gains and one major pre-existing base layer:

Layer
encoded hypothesis families
finite-class ERM kernel
finite uniform consistency bound
finite uniform recovery threshold
bit-family uniform recovery threshold
binary bit-family recovery threshold
finite uniform rate layer
finite uniform non-deceptive and recovery success bounds
Boolean quantitative bit-family non-deceptive, positive-margin, and recovery bounds
clocked $Kpoly$ construction, realization interface, and finite success-rate/good-sample bridge
clocked $Kpoly$ gap assessment: payload equals finite predictor-image obligation
clocked $Kpoly$ exact ZAB ERM closure and bare-interface obstruction
actual switched local normal form: ZAB ERM closure and locality-only obstruction
actual switched local strengthened structure: bounded-code and ZAB decision-list finite-image closure plus full-
actual switched local ERM construction: concrete $zOf, aOf, bOf, zfeat, samples$ wrapper
actual switched local code construction: concrete $zOf, aOf, bOf, zfeat, codes$ wrapper
actual switched local plug-in lookup endpoint
actual switched local plug-in majority endpoint
actual switched local finite-sample plug-in majority endpoint
actual switched local bounded-sample plug-in majority endpoint
finite weighted one-predictor agreement law
finite weighted family union bound
finite weighted recovery bounds
same-route switching/ERM certificate
same-route epsilon recovery certificate
atomic bad-code agreement obstruction
exact post-switch finite-summary kernel
exact visible predictor-image budget equivalence
reduced raw visible surface (a, b)
affine GF(2)-style column candidate class
bounded affine-feature candidate class
sparse-threshold affine candidate class
affine decision-list candidate class
exact-surface affine recovery ladder
shared-basis affine family ladder
raw exact-surface decision-list family on (a, b)
pullback route from reduced (a, b) families
raw (a, b) decision-list strict-subclass obstruction
affine candidate ladder on raw (a, b) bits
shared-basis affine ladder on raw (a, b) bits
quotient-route transfer to shared raw (a, b) ladders
shared raw (a, b) strict-subclass and exact-pullback obstruction
unified shared raw (a, b) target interface
canonical raw (a, b) specialization of the target interface
canonical raw (a, b) candidate-data interface with recovery consequence
explicit canonical raw (a, b) code-witness reduction
final canonical raw (a, b) recovery interface
ERM transport layer for fixed bit-encoded local classes
exact decision-list family on $(zfeat(z), a, b)$
shared $(zfeat(z), a, b)$ target and recovery interfaces with finite-image presentations
ERM specialization of the shared $(zfeat(z), a, b)$ route with finite-image objects

This is not the final proof. It is the right sort of groundwork.

20 Conclusion

The grassroots program should stay separate from the crux program.

The crux note asks where the present paper overreaches. This note asks what general machinery we would still want even if the overreach is patched later. On that constructive side, the current Lean result is that small encoded hypothesis families, finite empirical-risk minimizers, finite deceptive-sample bounds, finite uniform recovery thresholds, rational exact-recovery rates, explicit miss-ratio success bounds, weighted one-predictor agreement laws, weighted family deceptive-mass bounds, weighted exact recovery bounds, same-route epsilon recovery certificates, exact post-switch finite-summary compression kernels, one concrete affine GF(2)-style candidate family, one bounded affine-feature candidate family, and observer-relative input surfaces can already be handled cleanly inside one coherent Mettapedia/mathlib formal background layer. Optimistically, the same is now becoming true for the observational-semantics side: finite-depth HML distinction, weakness on operational quotients, and a concrete rho-calculus minimal-context instance are landing as reusable infrastructure rather than as one-off prose. But the current Meredith bridge is now also honest about its own shape: it produces global evidence summaries, not yet a nontrivial per-state learning surface. And even once one moves from those global summaries to the HML/context layer, the current observable family is infinitary by default and already concretely infinitary in the rho instance.

That is enough to sharpen the next optimistic task. We should not try to prove $P \neq NP$ in one jump. We should first prove that the intended switched predictors live in a genuinely small encoded class. The current best concrete landing point is now sharper than the earlier raw (a, b) route and sharper than the first direct $(\mathsf{zfeat}(z), a, b)$ route: prove that the real wrapper operates inside one shared-basis decision-list class on $(\mathsf{zfeat}(z), a, b)$, and then prove the corresponding bad-code agreement bound. The direct $r + 2k + 1$ route remains available, but the shared-basis $p + 1$ route is now the best optimistic target. Still, the direct route is no longer vague: it has been reduced all the way to an explicit code assignment plus an agreement bound. Only after one of those routes is instantiated should the rest of the argument be lifted on top of the foundation.